

Asset Disposal Module

Overview

The Asset Disposal Module provides a controlled process for requesting, approving, and completing the disposal of assets.

The module ensures that an asset cannot be disposed of simply because a disposal request has been created. A disposal request must first be approved by an authorized administrator. Only after approval may the requestor or authorized user complete the disposal.

The solution follows the same event-driven serverless architecture and progressive-enrichment data pattern used by the Assets Transfer Module.

The module supports:

- Single-asset disposal requests
- Multiple-asset disposal requests
- Approval or rejection
- Approval reminders and expiry
- Cancellation before disposal
- Disposal confirmation after approval
- Asset status updates in `assets_table`
- Audit history
- In-app notifications
- Email notifications
- Attachment and image storage in S3

Problem

Assets may become obsolete, damaged, uneconomical to repair, redundant, or otherwise unsuitable for continued use.

Without a controlled disposal workflow:

- Assets could be disposed of without authorization.
- There may be limited evidence of who requested or approved disposal.
- The asset register may continue to show an asset as active after physical disposal.
- Disposal documentation may be difficult to trace.
- Multiple assets may be disposed of without a consistent audit trail.
- Compliance and financial reporting may require manual investigation.

The organization requires a process that provides accountability from the initial disposal request through approval and final disposal.

Solution

The Asset Disposal Module introduces a controlled workflow consisting of:

1. Disposal Request Submission
2. Administrative Approval or Rejection
3. Disposal Cancellation or Expiry where applicable
4. Physical Disposal
5. Disposal Confirmation
6. Asset Record Update

A disposal request is created with status:

PENDING

Approval does **not** dispose of the asset.

Approval only authorizes the disposal.

The asset remains active in `assets_table` until the disposal action has been completed.

Once an approved disposal is physically carried out, the disposal request changes to:

APPROVED → DISPOSED

At this point the asset record is updated to indicate that the asset has been disposed of.

Valid Status Transitions

PENDING → APPROVED
PENDING → REJECTED
PENDING → EXPIRED
PENDING → CANCELLED

APPROVED → DISPOSED
APPROVED → CANCELLED

Invalid transitions include:

PENDING → DISPOSED X
REJECTED → APPROVED X
EXPIRED → APPROVED X
CANCELLED → APPROVED X
DISPOSED → APPROVED X
DISPOSED → CANCELLED X

User Experience

Requestor

The requestor can:

- Create a disposal request
- Select one or more eligible assets
- Provide a disposal reason
- Provide a description
- Provide supporting images or documentation
- Track the disposal request
- Cancel a pending or approved request where permitted
- Complete an approved disposal
- View the disposal history

Typical workflow:

```
Create Disposal Request
    ↓
Await Approval
    ↓
Receive Approval / Rejection Notification
    ↓
If Approved
    ↓
Physically Dispose of Asset(s)
    ↓
Confirm Disposal
    ↓
Asset Record Updated
```

Administrator

The administrator can:

- Review pending disposal requests
- Review all assets included in a request
- Approve disposal requests
- Reject disposal requests
- Provide approval comments where required
- Receive reminder notifications for pending requests
- Review disposal history

Typical workflow:

```
Receive Notification
    ↓
```

Review Disposal Request



Approve or Reject

Disposal Operator / Authorized User

The authorized user can:

- View approved disposal requests
- Confirm that the physical disposal has occurred
- Record the disposal method
- Record the disposal date
- Upload disposal evidence
- Upload disposal images
- Provide disposal notes
- Complete the disposal request

Typical workflow:

Receive Approved Request



Physically Dispose of Asset(s)



Record Disposal Details



Confirm Disposal



Asset Status Updated

Scope

Included

- Asset disposal requests
- Multiple assets per disposal request
- Approval workflow
- Approval reminders
- Approval expiry
- Disposal rejection workflow
- Disposal cancellation
- Disposal confirmation
- Disposal method and evidence
- Asset status updates
- Disposal audit history
- S3 attachment storage
- In-app notification delivery via `notifications_table`

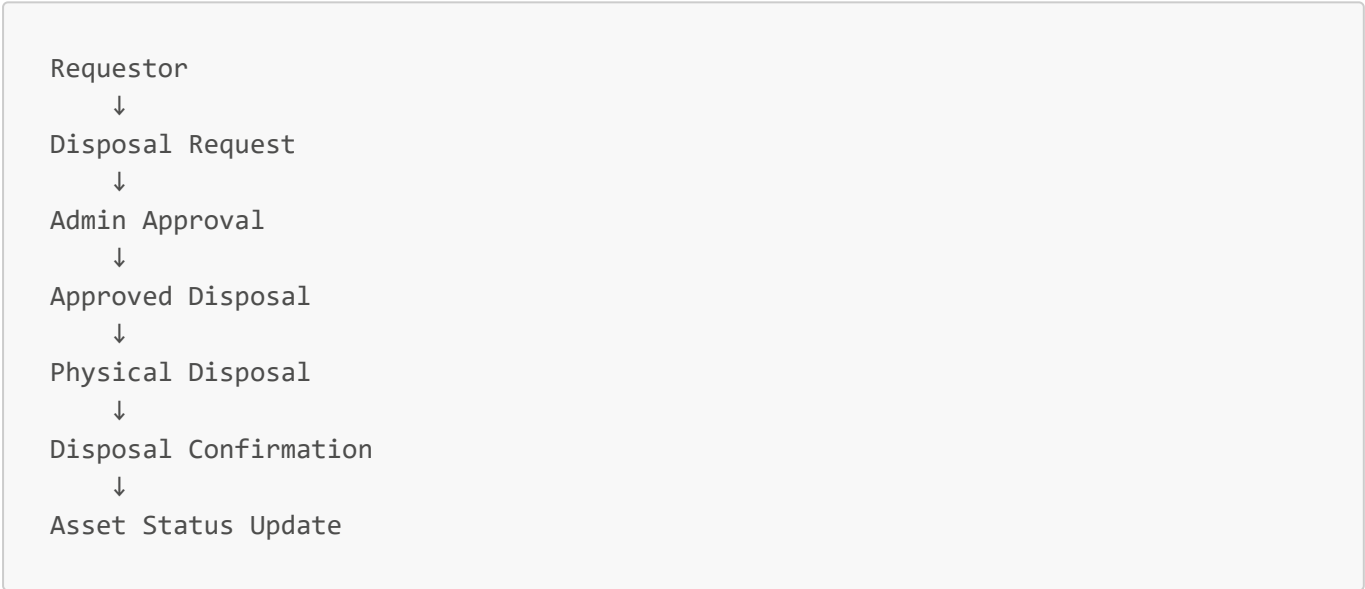
- Email notification delivery via SNS

Excluded

- Physical transportation of assets
- Asset procurement
- Asset transfer workflows
- Asset maintenance activities
- Financial accounting transactions
- Automated sale of assets
- Automated recycling vendor management
- Multi-stage approval chains

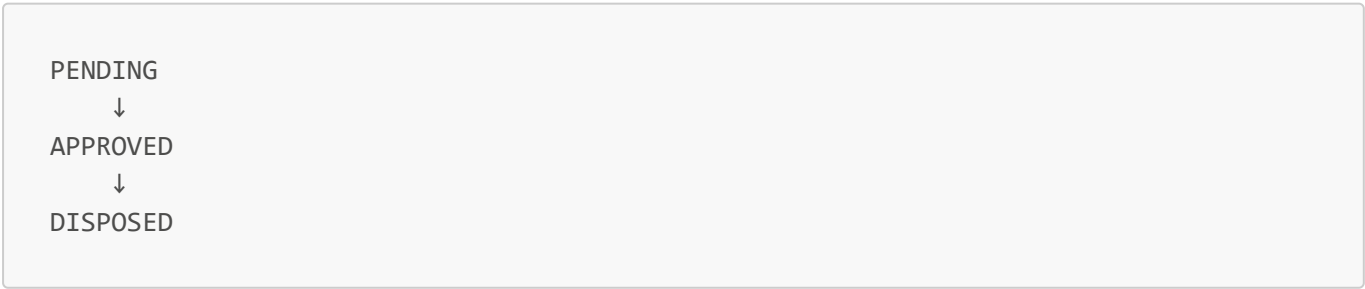
Architecture

High-Level Workflow



Disposal Status Lifecycle

Primary lifecycle:



Alternative outcomes:

PENDING
↓
REJECTED

PENDING
↓
EXPIRED

PENDING
↓
CANCELLED

APPROVED
↓
CANCELLED

State Transition Rules

All state transitions must be protected using DynamoDB conditional updates.

This ensures:

- An asset cannot be disposed of before approval.
- Duplicate requests cannot complete the same disposal twice.
- Invalid state transitions are rejected.
- Lambda retries do not result in inconsistent data.
- Only one successful disposal confirmation can update the asset.

Examples:

PENDING → APPROVED ✓
PENDING → REJECTED ✓
PENDING → EXPIRED ✓
PENDING → CANCELLED ✓

APPROVED → DISPOSED ✓
APPROVED → CANCELLED ✓

PENDING → DISPOSED ✗
REJECTED → APPROVED ✗
EXPIRED → APPROVED ✗
CANCELLED → APPROVED ✗

DISPOSED → APPROVED X
DISPOSED → CANCELLED X

Key Design Principles

Approval Does Not Dispose the Asset

Creating a disposal request does not change the asset.

Approving a disposal request also does not change the asset's operational status.

The asset remains active until the physical disposal has been completed and the disposal request transitions to:

DISPOSED

Only the disposal confirmation operation may update `assets_table`.

Multiple Assets

A single disposal request can contain multiple assets.

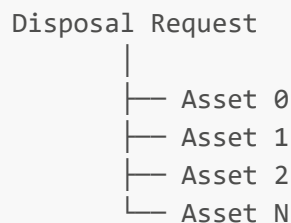
All assets are stored in the same `assets` array.

Example:

```
{
  "assets": [
    {
      "assetIndex": 0,
      "assetID": "RT-0013",
      "equipment": "Biltong Maker",
      "area": "processing",
      "images": []
    },
    {
      "assetIndex": 1,
      "assetID": "RT-0122",
      "equipment": "Vacuum Sausage Filler",
      "area": "processing",
      "images": []
    }
  ]
}
```

The request has one lifecycle and one approval decision.

Therefore:



Approval applies to the complete request.

Disposal confirmation applies to the complete request unless a future business requirement introduces partial disposal.

Important Rule

For the initial implementation, partial disposal is **not supported**.

Either:

ALL assets → DISPOSED

or the disposal request remains incomplete.

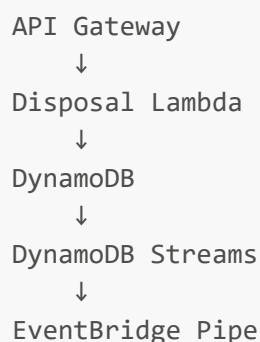
Event-Driven Processing

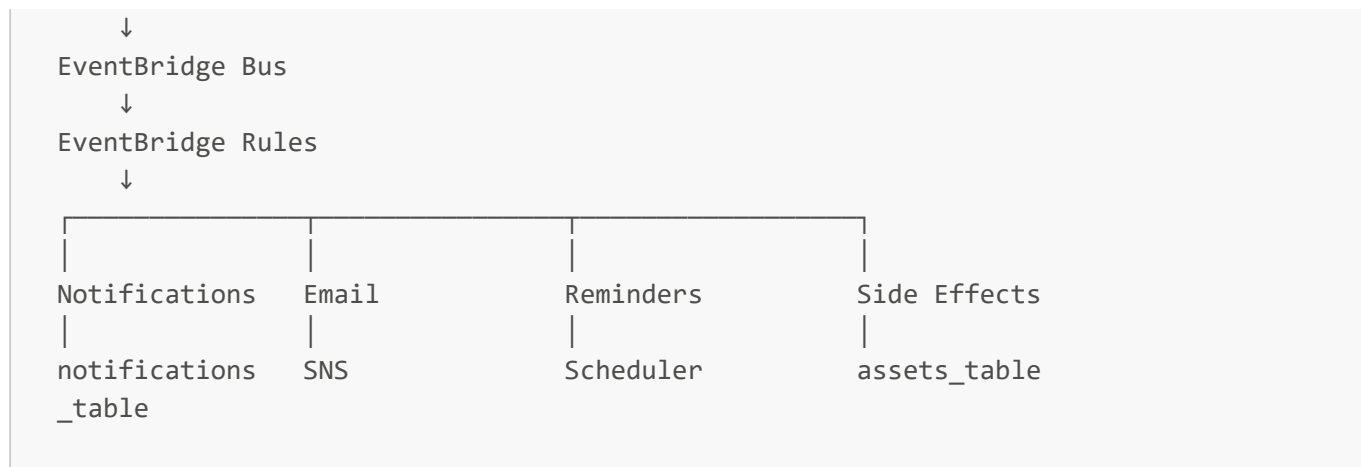
Writes to the API land directly in `asset_disposal_table`.

DynamoDB Streams capture `INSERT` and `MODIFY` events.

The stream feeds an EventBridge Pipe, which forwards events to the custom EventBridge bus.

EventBridge Rules pattern-match on the disposal status and route the event to the appropriate downstream Lambda.





The write path remains decoupled from notifications and other asynchronous side effects.

Notification Delivery

Two notification channels are maintained.

In-App Notifications

`handleNotifications` consumes disposal status events and writes notification records to:

```
notifications_table
```

Users retrieve notifications through:

```
GET /notifications
```

using the existing `getNotifications` Lambda.

Typical notification events:

- New disposal request
- Disposal approved
- Disposal rejected
- Disposal reminder
- Disposal expired
- Disposal cancelled
- Disposal completed

Email Notifications

Status-specific notification Lambdas publish messages to SNS topics.

Suggested notification functions:

```
assetDisposalRequest  
assetDisposalApproval  
assetDisposalCompletion
```

These notify the relevant users based on the disposal event.

Approval Timeouts

When a disposal request is created, an EventBridge Scheduler one-time schedule can be created for the configured approval period.

If the request remains:

```
PENDING
```

when the timeout occurs:

```
checkDisposalApprovalTimeout
```

checks the current status.

If the request is still pending:

```
PENDING → EXPIRED
```

The expiry event is then published through the normal EventBridge notification path.

If the request has already been approved, rejected, or cancelled, the scheduled action must not perform a state transition.

Scheduler Cleanup

The approval/rejection/cancellation path should delete the one-time scheduler once the request leaves **PENDING**.

This prevents stale schedules from accumulating.

Data Model

Single-Item, Progressive Enrichment Pattern

Each disposal request is represented by **one item** in `asset_disposal_table`.

The item is created when the request is submitted with:

```
status = PENDING
```

The item is then progressively enriched as the workflow advances.

Example:

```
PENDING
└─ pending

APPROVED
├─ pending
└─ approved

DISPOSED
├─ pending
├─ approved
└─ disposed
```

Rejected, cancelled, and expired requests retain the original request data and add their corresponding status block.

This keeps the complete disposal history and current state co-located on one DynamoDB item.

Data Ownership

Fields fall into two categories.

Client-Supplied

Examples:

- `assets`
- `description`
- `disposalReason`
- `location`
- `expectedDisposalDate`
- disposal details supplied during completion
- supporting attachments

Backend-Derived

Examples:

- requestorSub
- disposalCreated
- approvalId
- approvedDate
- approvedBySub
- disposalId
- disposedDate
- disposedBySub

Backend-derived fields must never be trusted from the client.

Lambda functions must obtain identity information from the authenticated Cognito claims and timestamps from the server.

asset_disposal_table

The following structure should be used for both single-asset and multiple-asset requests.

```
{
  "assetID": "string (PK / primary asset reference)",
  "id": "string (UUID)",
  "disposalCreated": "string (ISO 8601, backend-derived) (SK)",
  "status": "PENDING | APPROVED | REJECTED | EXPIRED | CANCELLED | DISPOSED",

  "requestorSub": "string (backend-derived from Cognito claim)",
  "approverSub": "string | null (intended/assigned approver)",
  "description": "string",
  "disposalReason": "string",
  "location": "string",
  "expectedDisposalDate": "string (ISO 8601, client-supplied)",
  "schedule_name": "string (disposal-id-timeout)",

  "assets": [
    {
      "assetIndex": "number",
      "assetID": "string",
      "equipment": "string",
      "area": "string",
      "assetIssueDetails": "string",
      "assetIssueReason": "string",
      "images": [
        {
          "bucket": "string",
          "key": "string",
          "filename": "string"
        }
      ]
    }
  ]
},
```

```
"pending": {
  "requestedBy": "string",
  "requestorName": "string",
  "requestorSub": "string",
  "description": "string",
  "disposalReason": "string",
  "location": "string",
  "expectedDisposalDate": "string"
},

"approved": {
  "approvalId": "string (UUID)",
  "approvedDate": "string (ISO 8601, backend-derived)",
  "approvedBy": "string",
  "approvedBySub": "string (backend-derived)",
  "approvalReminderCount": "number"
},

"disposed": {
  "disposalId": "string (UUID)",
  "disposedDate": "string (ISO 8601, backend-derived)",
  "disposedBy": "string",
  "disposedBySub": "string (backend-derived)",
  "disposalMethod": "string",
  "disposalLocation": "string | null",
  "disposalCost": "number | null",
  "disposalNotes": "string | null",
  "disposalImages": [
    {
      "bucket": "string",
      "key": "string",
      "filename": "string"
    }
  ],
  "disposalDocuments": [
    {
      "bucket": "string",
      "key": "string",
      "filename": "string"
    }
  ]
},

"cancelled": {
  "cancelledDate": "string (ISO 8601, backend-derived)",
  "cancelledBy": "string",
  "cancelledBySub": "string (backend-derived)",
  "cancelReason": "string"
},

"rejected": {
  "rejectedDate": "string (ISO 8601, backend-derived)",
  "rejectedBy": "string",
```

```

    "rejectedBySub": "string (backend-derived)",
    "rejectionReason": "string"
  },

  "expired": {
    "expiredDate": "string (ISO 8601, backend-derived)",
    "reason": "string"
  }
}

```

Multiple-Asset Example

The top-level request structure remains identical regardless of the number of assets.

```

{
  "id": "535be3f9-b1c1-4d35-8836-36c3b8d6c69d",
  "disposalCreated": "31 Aug 2026, 08:29",
  "status": "pending",

  "assets": [
    {
      "assetIndex": 0,
      "assetID": "RT-0013",
      "equipment": "Biltong Maker",
      "area": "processing",
      "assetIssueDetails": "",
      "assetIssueReason": "",
      "images": []
    },
    {
      "assetIndex": 1,
      "assetID": "RT-0122",
      "equipment": "Vacuum Sausage Filler",
      "area": "processing",
      "assetIssueDetails": "",
      "assetIssueReason": "",
      "images": []
    }
  ],

  "pending": {
    "requestedBy": "fabian petersen",
    "requestorName": "fabian",
    "requestorSub": "91bcf2e8-a091-705d-5642-00c963c96a50",
    "description": "Assets are no longer economical to maintain.",
    "disposalReason": "Beyond economical repair",
    "location": "distribution centre",
    "expectedDisposalDate": "31 Aug 2026"
  },

  "approved": null,

```

```
"disposed": null,  
"cancelled": null,  
"rejected": null,  
"expired": null  
}
```

After approval:

```
{  
  "id": "535be3f9-b1c1-4d35-8836-36c3b8d6c69d",  
  "disposalCreated": "31 Aug 2026, 08:29",  
  "status": "approved",  
  
  "assets": [  
    {  
      "assetIndex": 0,  
      "assetID": "RT-0013",  
      "equipment": "Biltong Maker",  
      "area": "processing",  
      "assetIssueDetails": "",  
      "assetIssueReason": "",  
      "images": []  
    },  
    {  
      "assetIndex": 1,  
      "assetID": "RT-0122",  
      "equipment": "Vacuum Sausage Filler",  
      "area": "processing",  
      "assetIssueDetails": "",  
      "assetIssueReason": "",  
      "images": []  
    }  
  ],  
  
  "pending": {  
    "requestedBy": "fabian petersen",  
    "requestorName": "fabian",  
    "requestorSub": "91bcf2e8-a091-705d-5642-00c963c96a50",  
    "description": "Assets are no longer economical to maintain.",  
    "disposalReason": "Beyond economical repair",  
    "location": "distribution centre",  
    "expectedDisposalDate": "31 Aug 2026"  
  },  
  
  "approved": {  
    "approvalId": "ff4a4739-d863-467f-a1bd-2d5f3c32383e",  
    "approvedDate": "31 Aug 2026, 08:40",  
    "approvedBy": "fabian petersen",  
    "approvedBySub": "91bcf2e8-a091-705d-5642-00c963c96a50",  
    "approvalReminderCount": 0  
  },  
}
```

```
"disposed": null,  
"cancelled": null,  
"rejected": null,  
"expired": null  
}
```

After disposal:

```
{  
  "id": "535be3f9-b1c1-4d35-8836-36c3b8d6c69d",  
  "disposalCreated": "31 Aug 2026, 08:29",  
  "status": "disposed",  
  
  "assets": [  
    {  
      "assetIndex": 0,  
      "assetID": "RT-0013",  
      "equipment": "Biltong Maker",  
      "area": "processing",  
      "assetIssueDetails": "",  
      "assetIssueReason": "",  
      "images": []  
    },  
    {  
      "assetIndex": 1,  
      "assetID": "RT-0122",  
      "equipment": "Vacuum Sausage Filler",  
      "area": "processing",  
      "assetIssueDetails": "",  
      "assetIssueReason": "",  
      "images": []  
    }  
  ],  
  
  "pending": {  
    "requestedBy": "fabian petersen",  
    "requestorName": "fabian",  
    "requestorSub": "91bcf2e8-a091-705d-5642-00c963c96a50",  
    "description": "Assets are no longer economical to maintain.",  
    "disposalReason": "Beyond economical repair",  
    "location": "distribution centre",  
    "expectedDisposalDate": "31 Aug 2026"  
  },  
  
  "approved": {  
    "approvalId": "ff4a4739-d863-467f-a1bd-2d5f3c32383e",  
    "approvedDate": "31 Aug 2026, 08:40",  
    "approvedBy": "fabian petersen",  
    "approvedBySub": "91bcf2e8-a091-705d-5642-00c963c96a50",  
    "approvalReminderCount": 0  
  },  
}
```



```
"disposed": {
  "disposalId": "a4f9a1c1-6f0d-4b21-8f7c-123456789abc",
  "disposedDate": "31 Aug 2026, 11:15",
  "disposedBy": "fabian petersen",
  "disposedBySub": "91bcf2e8-a091-705d-5642-00c963c96a50",
  "disposalMethod": "Recycled",
  "disposalLocation": "Approved recycling facility",
  "disposalCost": 250,
  "disposalNotes": "Asset physically removed and recycled.",
  "disposalImages": [],
  "disposalDocuments": []
},

"cancelled": null,
"rejected": null,
"expired": null
}
```

notifications_table

The existing notification table can be reused.

For disposal notifications, the notification should identify the disposal request and asset(s).

```
{
  "recipientSub": "string (PK)",
  "notificationCreated": "string (SK)",
  "id": "string",
  "disposalId": "string",
  "recipientEmail": "string",
  "assetId": "string | null",
  "type": "ASSET_DISPOSAL",
  "title": "string",
  "message": "string",
  "location": "string",
  "status": "UNREAD | READ | ARCHIVED",
  "priority": "LOW | NORMAL | HIGH | URGENT",
  "sub": "string (Cognito sub of recipient)",
  "channels": "IN_APP | EMAIL | PUSH | SMS",
  "dateRead": "string | null"
}
```

For a multi-asset disposal, `assetId` may either:

- contain the primary asset ID, or
- be `null` with the `disposalId` used to retrieve the complete request.

The disposal request itself remains the authoritative record.

API Design

Create Disposal Request

```
POST /asset-disposals
```

Creates a new disposal request.

Initial status:

```
PENDING
```

Request

```
{
  "assets": [
    {
      "assetID": "RT-0013",
      "assetIndex": 0
    },
    {
      "assetID": "RT-0122",
      "assetIndex": 1
    }
  ],
  "description": "Assets are no longer economical to maintain.",
  "disposalReason": "Beyond economical repair",
  "location": "distribution centre",
  "expectedDisposalDate": "2026-08-31"
}
```

The backend retrieves the current asset details from `assets_table` rather than trusting client-supplied equipment, area, or other authoritative asset attributes.

Approve / Reject Disposal

```
POST /asset-disposals/{disposalId}/approval
```

The administrator provides a decision:

```
{  
  "decision": "APPROVED"  
}
```

or:

```
{  
  "decision": "REJECTED",  
  "rejectionReason": "Asset should be repaired rather than disposed."  
}
```

The Lambda obtains the approving user's identity from Cognito.

Complete Disposal

```
POST /asset-disposals/{disposalId}/dispose
```

Only an approved request can be completed.

Example:

```
{  
  "disposalMethod": "Recycled",  
  "disposalLocation": "Approved recycling facility",  
  "disposalCost": 250,  
  "disposalNotes": "Asset physically removed and recycled.",  
  "disposalImages": [],  
  "disposalDocuments": []  
}
```

The Lambda must verify:

```
status == APPROVED
```

before performing the disposal.

The operation then atomically:

1. Updates the disposal request to **DISPOSED**.
2. Adds the **disposed** block.
3. Updates the relevant assets in **assets_table**.

4. Prevents a second disposal confirmation.

Cancel Disposal

```
POST /asset-disposals/{disposalId}/cancel
```

Cancellation is permitted only for configured states.

Initial implementation:

```
PENDING → CANCELLED  
APPROVED → CANCELLED
```

Example:

```
{  
  "cancelReason": "Asset will remain in service."  
}
```

Get Disposal Requests

```
GET /asset-disposals
```

Supports filtering by:

- status
- location
- requestor
- date
- asset ID

Example:

```
GET /asset-disposals?status=PENDING
```

Get Disposal Request

```
GET /asset-disposals/{disposalId}
```

Returns the complete progressive item, including all populated status blocks.

Frontend

Disposal Request Form

The frontend should follow the same form architecture as the transfer module.

Request Fields

```
Location
Disposal Reason
Expected Disposal Date
Description
Assets
```

Asset Selection

The form must support:

```
+ Add Asset
```

Multiple assets are represented as:

```
assets: [
  {
    assetID: string;
    assetIndex: number;
    equipment: string;
    area: string;
    assetIssueDetails: string;
    assetIssueReason: string;
    images: File[];
  }
]
```

The frontend should only allow assets eligible for disposal according to the current asset status and user permissions.

Disposal Details Form

When the request is **APPROVED**, the authorized user can complete the disposal.

Fields:

Disposal Method

Disposal Location

Disposal Cost

Disposal Notes

Disposal Images

Disposal Documents

The disposal action must not be available when:

PENDING

REJECTED

EXPIRED

CANCELLED

DISPOSED

It is available only when:

APPROVED

Table / List View

The disposal table should follow the transfer table pattern.

Suggested columns:

Column	Purpose
Disposal ID	Unique disposal request
Assets	Number / summary of assets
Location	Asset location
Reason	Disposal reason
Requestor	Person who requested disposal
Created	Request date

Column	Purpose
Expected Date	Planned disposal date
Status	Current workflow state
Actions	Available workflow actions

Multiple assets should be expandable rather than creating separate top-level rows.

Status-Based Actions

PENDING

Requestor:

View

Cancel

Administrator:

View

Approve

Reject

APPROVED

Requestor / authorized disposal user:

View

Dispose

Cancel

Administrator:

View

Cancel

DISPOSED

All authorized users:

[View](#)

No further workflow action is available.

REJECTED

[View](#)

EXPIRED

[View](#)

CANCELLED

[View](#)

Backend Lambda Functions

#	Lambda Function	Purpose
1	<code>postDisposalRequest</code>	Create disposal request with PENDING status
2	<code>postDisposalApproval</code>	Approve or reject disposal request
3	<code>postDisposal</code>	Complete approved disposal and update assets
4	<code>postDisposalCancel</code>	Cancel pending/approved disposal
5	<code>getDisposalList</code>	Retrieve disposal requests
6	<code>getDisposalById</code>	Retrieve one disposal request
7	<code>getNotifications</code>	Retrieve in-app notifications
8	<code>handleNotifications</code>	Write disposal notifications
9	<code>assetDisposalRequest</code>	Notify administrators of new request
10	<code>assetDisposalApproval</code>	Notify requestor of approval/rejection
11	<code>assetDisposalCompletion</code>	Notify relevant users of completed disposal
12	<code>checkDisposalApprovalTimeout</code>	Expire pending approval requests

Data Layer

Table	Purpose
<code>asset_disposal_table</code>	Disposal workflow and audit history
<code>assets_table</code>	Current asset information
<code>notifications_table</code>	In-app notifications
<code>users_table</code>	User details for notifications

Infrastructure

Frontend

Service	Purpose
CloudFront	Content delivery and secure access
S3	Static website hosting

Authentication

Service	Purpose
Cognito	Authentication and authorization

API Layer

Service	Purpose
API Gateway	Secure API endpoints

Compute

Service	Purpose
Lambda	Request, approval, disposal, cancellation and notification processing

Storage

Service	Purpose
DynamoDB	Disposal workflow, assets and notifications
S3	Disposal images and supporting documents

Event Processing

Service	Purpose
DynamoDB Streams	Captures disposal table changes
EventBridge Pipes	Routes stream events
EventBridge	Custom event bus
EventBridge Rules	Status-based event fan-out
EventBridge Scheduler	Approval timeout
SQS	Retry buffering where required
DLQ	Failed message handling
SNS	Email notification delivery

Asset Table Update

The disposal module must not update the asset record when:

PENDING

or:

APPROVED

The asset is updated only when:

APPROVED → DISPOSED

The exact asset status attribute should follow the existing Asset Management Module convention.

Conceptually:

```
{
  "assetID": "RT-0013",
  "status": "DISPOSED",
  "disposalId": "535be3f9-b1c1-4d35-8836-36c3b8d6c69d",
  "dateDisposed": "2026-08-31T11:15:00+02:00"
}
```

For multiple assets, every asset in the request must be updated.

The update should use conditional writes so that an asset cannot be incorrectly changed if its state has changed since the disposal request was created.

S3 File Structure

Disposal attachments should follow the same predictable structure used by the transfer module.

Suggested structure:

```
disposals/  
  {disposalId}/  
    assets/  
      {assetIndex}/  
        images/  
          image.webp  
    documents/  
      document.pdf  
    disposal/  
      images/  
        image.webp  
      documents/  
        disposal-document.pdf
```

This keeps files grouped by disposal request and asset index.

Security and Authorization

Cognito groups and claims must control access.

Suggested roles:

```
User  
Manager  
Admin
```

Request Creation

Users may request disposal only for assets they are authorized to manage.

Approval

Only authorized administrators may approve or reject disposal requests.

Disposal

Only authorized users may complete an approved disposal.

The backend must enforce these rules.

The frontend must not be treated as the security boundary.

Auditability

Every disposal request must retain:

- Requestor
- Request date
- Assets included
- Disposal reason
- Original asset location
- Expected disposal date
- Approver
- Approval date
- Approval decision
- Disposal operator
- Disposal date
- Disposal method
- Disposal evidence
- Cancellation details
- Rejection details
- Expiry details

The disposal item therefore acts as the complete audit record.

Idempotency

Asynchronous EventBridge, SQS and Lambda processing may result in retries.

Notification Lambdas should use an idempotency key such as:

```
disposalId#status
```

before creating notifications or publishing to SNS.

The disposal completion Lambda must also prevent duplicate completion.

The DynamoDB update should include a conditional expression equivalent to:

```
status = APPROVED
```

so a second completion attempt fails safely.

Fault Tolerance

Where asynchronous processing requires buffering, use:

```
SQS
  ↓
Lambda
  ↓
DLQ
```

with:

```
maxReceiveCount = 3
```

CloudWatch alarms should monitor:

```
ApproximateNumberOfMessagesVisible
```

on the DLQ.

Observability

Every disposal event should carry a correlation identifier.

Recommended event detail:

```
{
  "disposalId": "string",
  "status": "APPROVED",
  "correlationId": "disposalId",
  "timestamp": "ISO 8601"
}
```

The correlation ID should remain consistent through:

```
API
→ Lambda
→ DynamoDB
→ DynamoDB Stream
→ EventBridge
→ Lambda
→ SNS
```

CloudWatch should provide:

- Lambda error monitoring
- EventBridge failures
- Scheduler failures
- SQS queue depth
- DLQ alarms
- Disposal completion errors

AWS X-Ray active tracing may be enabled for the synchronous API path.

Prerequisites

The following components must exist before the module can operate:

- Asset Management Module
- Asset Registry (`assets_table`)
- Cognito authentication
- User roles and permissions
- S3 asset storage
- EventBridge infrastructure
- DynamoDB Streams
- Notification infrastructure
- SNS topics/subscriptions
- SQS/DLQ infrastructure where required

Required roles:

```
Requestor
Administrator
Authorized Disposal User
```

Business Rules

Eligible Assets

The system must define which asset statuses are eligible for disposal.

Examples of potentially eligible assets:

```
ACTIVE
DAMAGED
BEYOND_REPAIR
OBSOLETE
```

The exact eligible statuses must be confirmed against the Asset Management Module.

Asset Location

The asset's current location should be captured when the disposal request is created.

The backend should validate that the asset still exists and remains eligible before approval and again before final disposal.

Asset Changed After Request

If the asset has materially changed between request creation and disposal, the disposal Lambda should revalidate the asset before completing the disposal.

Examples:

```
Asset transferred
Asset already disposed
Asset deleted
Asset status changed
```

In these cases disposal completion should fail safely rather than updating an unexpected asset.

Open Questions

Business Questions

- Which asset statuses are eligible for disposal?
- Who can create disposal requests?
- Who can approve disposal requests?
- Who can perform the physical disposal?
- Can an administrator dispose of an asset directly without a request?
- What disposal methods are allowed?
- Is a disposal document mandatory?
- Are disposal images mandatory?
- Is a disposal cost required?

- Is a disposal location required?
- What is the approval expiry period?
- How many approval reminders should be sent?
- Can an approved disposal be cancelled?
- What happens if only some assets in a multi-asset request can be disposed of?
- Should disposal approval require comments?
- Should financial/book value information be included?

Technical Questions

- Should `asset_disposal_table` use `assetID` as the partition key in the same way as the transfer table, or should `id/disposalId` become the primary request key?
- Should disposal history be immutable?
- Should disposal events be exposed for reporting integrations?
- Should disposal notifications use SQS + DLQ for all email paths?
- Should disposal evidence be immutable after completion?
- Should an idempotency table be introduced for event processing?
- Should the asset update and disposal status update be performed using a DynamoDB transaction?

Future Enhancements

- Multi-stage disposal approval
- Financial/book-value integration
- Automated disposal vendor management
- Recycling vendor integration
- Disposal certificate generation
- Disposal analytics dashboard
- QR-code disposal confirmation
- Mobile disposal confirmation
- Bulk disposal campaigns
- Integration with procurement and finance systems
- Integration with maintenance history
- Automated disposal eligibility recommendations

Architecture Review Notes & Recommendations

1. Use DynamoDB Transactions for Final Disposal

The final disposal operation has two critical changes:

```
asset_disposal_table → DISPOSED  
assets_table → DISPOSED
```


These changes should ideally be performed using `TransactWriteItems` where the DynamoDB key design allows it.

This prevents the disposal workflow from becoming `DISPOSED` while the asset record remains active, or vice versa.

2. Revalidate Assets Before Disposal

The assets selected when the request was created may no longer have the same state when disposal is approved.

Before final disposal:

```
Read current asset
  ↓
Validate eligibility
  ↓
Validate ownership/location/state
  ↓
Complete disposal
```

This prevents stale disposal requests from modifying an asset incorrectly.

3. Preserve the Complete Request Structure

The API response should maintain the same top-level structure regardless of status:

```
id
disposalCreated
status
assets
pending
approved
disposed
cancelled
rejected
expired
```

Unused lifecycle blocks should be:

```
null
```

This makes the frontend predictable and allows the same table/card/detail components to render every status.

4. Keep Multiple Assets in One Request

The initial implementation should use:

```
one disposal request
    ↓
one DynamoDB item
    ↓
many assets[]
```

rather than creating a separate disposal workflow for every asset.

This matches the existing transfer pattern and makes a single approval decision applicable to the complete request.

5. Avoid Partial Disposal Initially

Partial disposal introduces substantially more state complexity.

For the first version:

```
APPROVED → DISPOSED
```

must mean every asset in the request has been disposed of.

If partial disposal is required later, it can be introduced as a separate workflow/state model.